

SIMATS ENGINEERING
CO4-ASSESSMENT TOOL3- INTEGRATED EXPERIMENTS

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Name	N.Sai Sankar
Register Number	192311187

COURSE OUTCOME COVERED

CO4: Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)

Assessment Tool Weightage: 40%

Time Duration: 45 Minutes

QUESTIONS: 2

Total Marks:20

Code of Conduct:

I N.Sai Sankar(192311187) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

- (a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.
- (b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification
- (c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

- (a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, -1 each step, -5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α , γ , ϵ).
- (b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.
- (c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

(a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.

Program:-

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

# Load Iris dataset
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(
    iris.data,
    columns=iris.feature_names
)

df["target"] = iris.target

# Display first 5 rows
print("First 5 rows:")
print(df.head())

# Display data types
print("\nData Types:")
print(df.dtypes)

# Check missing values
print("\nMissing Values:")
print(df.isnull().sum())

# Separate features and target
```

```

X = df.drop("target", axis=1)
y = df["target"]

# Split into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))

```

Output:

```

>> ===== RESTART: C:/Users/MY PC/OneDrive/Documents/CSA1709-AI/Expl a.py =====
First 5 rows:
   sepal length (cm)  sepal width (cm)  ...  petal width (cm)  target
0                5.1                3.5  ...                0.2        0
1                4.9                3.0  ...                0.2        0
2                4.7                3.2  ...                0.2        0
3                4.6                3.1  ...                0.2        0
4                5.0                3.6  ...                0.2        0

[5 rows x 5 columns]

Data Types:
sepal length (cm)    float64
sepal width (cm)     float64
petal length (cm)    float64
petal width (cm)     float64
target              int64
dtype: object

Missing Values:
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
target              0
dtype: int64

Training samples: 120
Testing samples: 30
>>

```

(b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification

Program:-

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

# Load Iris dataset
iris = load_iris()

# Input features and target
X = iris.data
y = iris.target

# Split dataset into training and testing data
x_train, x_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# Create Decision Tree
model = DecisionTreeClassifier(
    criterion="gini",
    random_state=42
)

# Train model
model.fit(x_train, y_train)

# Print tree depth
print("Tree Depth:", model.get_depth())

# Print number of leaves
```

```
print("Number of Leaves:", model.get_n_leaves())
# Feature importance
print("\nFeature Importance:")

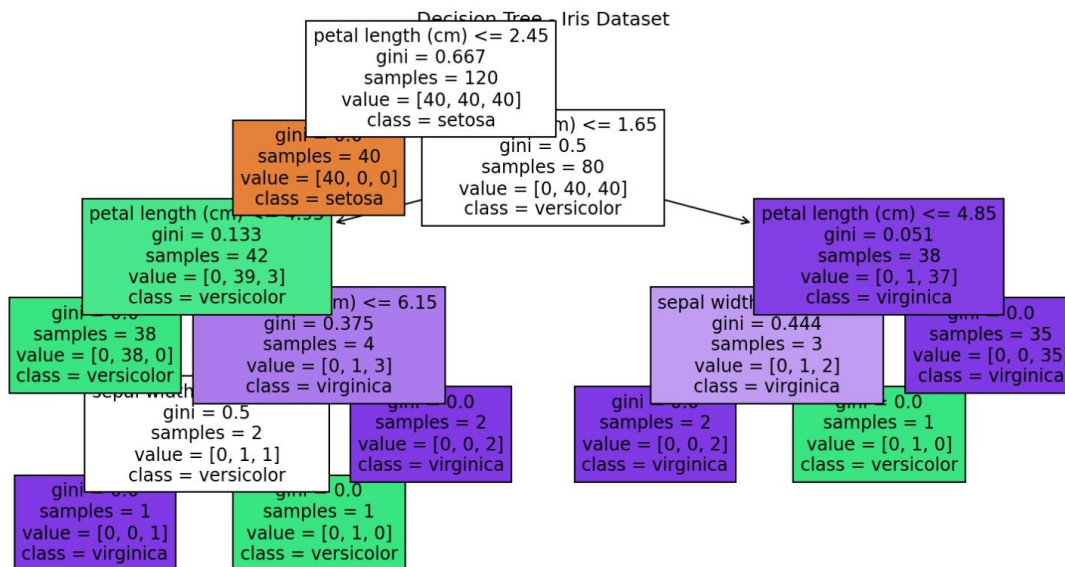
for feature, importance in zip(
    iris.feature_names,
    model.feature_importances_
):
    print(feature, ":", importance)
# Identify root node
root_index = model.tree_.feature[0]
root_feature = iris.feature_names[root_index]
print("\nRoot Node:", root_feature)
# Plot decision tree
plt.figure(figsize=(15, 8))

plot_tree(
    model,
    feature_names=iris.feature_names,
    class_names=iris.target_names,
    filled=True
)

plt.title("Decision Tree - Iris Dataset")

plt.show()
```

Output:



Root Node

For the Iris dataset, the root node will generally be based on:

Petal length (cm)

or another highly informative petal feature depending on the exact training split/model configuration.

Justification

The root node is selected because it provides the greatest reduction in impurity according to the selected splitting criterion.

The Gini impurity is:

$$Gini = 1 - \sum_{i=1}^n p_i^2$$

where p_i is the probability of a sample belonging to class i .

(c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Program:-

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)
# ----- (a) Load, preprocess, split -----
iris = load_iris()
X = iris.data
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
# ----- (b) Build the Decision Tree model -----
```



```

model = DecisionTreeClassifier(criterion="gini", random_state=42, max_depth=4)
model.fit(X_train, y_train)
print("Tree structure:")
print(export_text(model, feature_names=list(iris.feature_names)))
# ----- (c) Evaluate -----
# Make predictions
y_pred = model.predict(X_test)
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:", accuracy)
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(cm)
# Classification Report
print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=iris.target_names
    )
)
# Find misclassified instances
mask = y_test != y_pred
misclassified = pd.DataFrame(X_test[mask], columns=iris.feature_names)
misclassified["Actual"] = [iris.target_names[i] for i in y_test[mask]]
misclassified["Predicted"] = [iris.target_names[i] for i in y_pred[mask]]
print("\nMisclassified Instances:")
print(misclassified)

```

Output:

```

>
===== RESTART: C:/Users/MY PC/OneDrive/Documents/CSA1709-AI/CO4/EXPl c.py =====
Tree structure:
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) > 2.45
|   |--- petal width (cm) <= 1.65
|   |   |--- petal length (cm) <= 4.95
|   |   |   |--- class: 1
|   |   |   |--- petal length (cm) > 4.95
|   |   |       |--- sepal length (cm) <= 6.15
|   |   |       |   |--- class: 1
|   |   |       |   |--- sepal length (cm) > 6.15
|   |   |           |--- class: 2
|   |   |--- petal width (cm) > 1.65
|   |       |--- petal length (cm) <= 4.85
|   |       |   |--- sepal width (cm) <= 3.00
|   |       |   |   |--- class: 2
|   |       |   |   |--- sepal width (cm) > 3.00
|   |       |       |--- class: 1
|   |       |--- petal length (cm) > 4.85
|   |           |--- class: 2
|   |--- class: 2

Accuracy: 0.9333333333333333

Confusion Matrix:
[[10  0  0]
 [ 0  9  1]
 [ 0  1  9]]

Classification Report:

```

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	0.90	0.90	0.90	10
virginica	0.90	0.90	0.90	10
accuracy			0.93	30
macro avg	0.93	0.93	0.93	30
weighted avg	0.93	0.93	0.93	30

EXPERIMENT 2: REINFORCEMENT LEARNING – Q-LEARNING

(a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, -1 each step, -5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α , γ , ϵ).

Environment

We use a 4×4 grid.

There are:

- 16 states
- Actions:
 - Up
 - Down
 - Left
 - Right
- Goal reward = +10
- Each normal step = -1
- Obstacle = -5

Program:-

```
import numpy as np
import random
import matplotlib.pyplot as plt
# Grid dimensions
ROWS = 4
COLS = 4
# States
START = (0, 0)
GOAL = (3, 3)
# Obstacles
OBSTACLES = [(1, 1), (2, 2)]
# Actions
ACTIONS = {
    0: (-1, 0), # Up
    1: (1, 0), # Down
    2: (0, -1), # Left
    3: (0, 1) # Right
}
ACTION_NAMES = {
    0: "U",
    1: "D",
    2: "L",
    3: "R"
}
# Hyperparameters
alpha = 0.1
gamma = 0.9
epsilon = 0.1
episodes = 100
# Q-table
Q = np.zeros((ROWS * COLS, 4))
# Convert position to state number
def state_number(position):
```

```

    return position[0] * COLS + position[1]
# Perform action
def step(state, action):
    r, c = state
    dr, dc = ACTIONS[action]
    new_r = r + dr
    new_c = c + dc
    # Boundary checking
    if new_r < 0 or new_r >= ROWS or new_c < 0 or new_c >= COLS:
        return state, -1, False
    new_state = (new_r, new_c)
    # Obstacle
    if new_state in OBSTACLES:
        return state, -5, False
    # Goal
    if new_state == GOAL:
        return new_state, 10, True
    # Normal movement
    return new_state, -1, False
# Epsilon-greedy action selection
def choose_action(state):
    s = state_number(state)
    if random.random() < epsilon:
        return random.randint(0, 3)
    return np.argmax(Q[s])
# Store cumulative rewards
rewards_per_episode = []
# Q-learning
for episode in range(1, episodes + 1):
    state = START
    total_reward = 0
    for step_count in range(100):
        action = choose_action(state)
        next_state, reward, done = step(state, action)
        s = state_number(state)

```

```

ns = state_number(next_state)
# Q-learning update
Q[s, action] = Q[s, action] + alpha * (
    reward +
    gamma * np.max(Q[ns]) -
    Q[s, action]
)
total_reward += reward
state = next_state
if done:
    break
rewards_per_episode.append(total_reward)
# Display Q-values
print("Final Q-table:")
print(np.round(Q, 2))
# Display policy
print("\nLearned Policy:")
for r in range(ROWS):
    row = []
    for c in range(COLS):
        state = (r, c)
        if state == GOAL:
            row.append("G")
        elif state in OBSTACLES:
            row.append("X")
        else:
            s = state_number(state)
            best_action = np.argmax(Q[s])
            row.append(ACTION_NAMES[best_action])
    print(row)
# Plot cumulative reward
plt.figure(figsize=(10, 5))
plt.plot(rewards_per_episode)
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")

```

```
plt.title("Q-Learning Cumulative Reward")
plt.grid(True)
plt.show()
```

Output:-

```
> ===== RESTART: C:/Users/MY PC/OneDrive/Documents/CSA1709-AI/CO4/EXP2 a.py =====
Final Q-table:
[[-2.29 -2.34 -2.32  0.44]
 [-1.73 -2.21 -1.75  2.25]
 [-0.09 -0.8  -1.22  4.14]
 [-0.13  6.04 -0.49  0. ]
 [-1.79 -1.56 -1.81 -2.4 ]
 [ 0.    0.    0.    0. ]
 [-0.5  -0.98 -0.5   1.54]
 [-0.31  7.97 -0.48 -0.2 ]
 [-1.03 -0.96 -0.96 -0.12]
 [-0.5   2.43 -0.52 -1.6 ]
 [ 0.    0.    0.    0. ]
 [ 0.52 10.    1.07  2.13]
 [-0.51 -0.58 -0.59 -0.04]
 [-0.3  -0.2  -0.3   5.96]
 [-0.89  0.18 -0.11  9.42]
 [ 0.    0.    0.    0. ]]

Learned Policy:
['R', 'R', 'R', 'D']
['D', 'X', 'R', 'D']
['R', 'D', 'X', 'D']
['R', 'R', 'R', 'G']
>
```

(b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.

Program:-

```
import numpy as np
import random
# Grid World
N = 4
start, goal = (0, 0), (3, 3)
obstacles = {(1, 1), (2, 2)}
moves = [(-1,0), (1,0), (0,-1), (0,1)]
actions = ["Up", "Down", "Left", "Right"]
# Parameters
alpha, gamma = 0.1, 0.9
epsilon, decay, eps_min = 1.0, 0.98, 0.05
episodes, max_steps = 100, 100
Q = np.zeros((N, N, 4))
def step(s, a):
    r, c = s
    nr, nc = r + moves[a][0], c + moves[a][1]
    if not (0 <= nr < N and 0 <= nc < N):
        return s, -1, False
    ns = (nr, nc)
    if ns in obstacles:
        return s, -5, False
    if ns == goal:
        return ns, 10, True
    return ns, -1, False
# Q-Learning
for ep in range(episodes):
    state = start
    for _ in range(max_steps):
        action = (random.randrange(4) if random.random() < epsilon
```

```

else np.argmax(Q[state]))
next_state, reward, done = step(state, action)
target = reward if done else reward + gamma * np.max(Q[next_state])
Q[state[0], state[1], action] += alpha * (
target - Q[state[0], state[1], action]
)
state = next_state
if done:
break
epsilon = max(eps_min, epsilon * decay)
# Final Q-table
print("Final Q-Table:")
for r in range(N):
for c in range(N):
print((r, c), np.round(Q[r, c], 2))

```

Output:-

```

Final Q-Table:
(0, 0) [-2.12  1.08 -2.39 -2.55]
(0, 1) [-2.02 -4.41 -2.28 -1.99]
(0, 2) [-1.55 -1.26 -1.9  -1.13]
(0, 3) [-1.05  0.88 -1.27 -1.14]
(1, 0) [-2.46  2.7  -1.7  -5.07]
(1, 1) [0.  0.  0.  0.]
(1, 2) [-1.2  -3.33 -4.79 -0.02]
(1, 3) [-1.11  4.79 -1.08 -0.01]
(2, 0) [-0.99  4.37 -1.1  0.64]
(2, 1) [-3.17  4.84 -0.92 -3.27]
(2, 2) [0.  0.  0.  0.]
(2, 3) [ 0.7  8.5  -0.68  1.44]
(3, 0) [-0.29  1.65  0.15  6.13]
(3, 1) [0.7  1.11  1.41  7.98]
(3, 2) [ 1.12  5.46  2.24 10.  ]
(3, 3) [0.  0.  0.  0.]
>|

```


(C) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

Program:-

```
import numpy as np
import random
import matplotlib.pyplot as plt
N = 4
start, goal = (0, 0), (3, 3)
obstacles = {(1, 1), (2, 2)}
moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
symbols = ["↑", "↓", "←", "→"]
alpha, gamma = 0.1, 0.9
epsilon, decay, eps_min = 1.0, 0.98, 0.05
episodes, max_steps = 100, 100
Q = np.zeros((N, N, 4))
rewards = []
def step(state, action):
    r, c = state
    nr = r + moves[action][0]
    nc = c + moves[action][1]
    if not (0 <= nr < N and 0 <= nc < N):
        return state, -1, False
    next_state = (nr, nc)
    if next_state in obstacles:
```

```

return state, -5, False
if next_state == goal:
return next_state, 10, True
return next_state, -1, False
for episode in range(epochs):
state = start
total_reward = 0
for _ in range(max_steps):
if random.random() < epsilon:
action = random.randrange(4)
else:
action = np.argmax(Q[state])
next_state, reward, done = step(state, action)
target = reward if done else (
reward + gamma * np.max(Q[next_state])
)
Q[state[0], state[1], action] += alpha * (
target - Q[state[0], state[1], action]
)
state = next_state
total_reward += reward
if done:
break
rewards.append(total_reward)
epsilon = max(eps_min, epsilon * decay)
print("\nFINAL LEARNED POLICY")
print("-----")
for r in range(N):
for c in range(N):
state = (r, c)
if state == start:
symbol = "S"
elif state == goal:
symbol = "G"
elif state in obstacles:

```

```

symbol = "X"
else:
symbol = symbols[np.argmax(Q[state])]
print(symbol, end=" ")
print()
print("\nFINAL Q-TABLE")
print("-----")
for r in range(N):
    for c in range(N):
        print(
            f"State ({r},{c}):",
            np.round(Q[r, c], 2)
        )
window = 10
average = np.convolve(
    rewards,
    np.ones(window) / window,
    mode="valid"
)
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(range(1, episodes + 1), rewards)
plt.xlabel("Episode")
plt.ylabel("Reward")
plt.title("Q-Learning Reward")
plt.grid()
plt.subplot(1, 2, 2)
plt.plot(range(window, episodes + 1), average)
plt.xlabel("Episode")
plt.ylabel("Average Reward")
plt.title("Q-Learning Convergence")
plt.grid()
plt.tight_layout()
plt.show()

```

Output:-

FINAL LEARNED POLICY

S → ↓ ↓
↓ X → ↓
↓ ↓ X ↓
→ → → G

FINAL Q-TABLE

State (0,0): [-2.51 -2.31 -2.13 0.72]
State (0,1): [-0.93 -5.2 -2.05 2.47]
State (0,2): [-1.01 4.26 -1.27 -0.05]
State (0,3): [-0.94 3.64 -1.35 -0.89]
State (1,0): [-1.95 -1.41 -1.79 -5.15]
State (1,1): [0. 0. 0. 0.]
State (1,2): [-0.41 -2.2 -2.24 6.09]
State (1,3): [-0.51 7.97 1.61 2.43]
State (2,0): [-1.62 0.17 -1.11 -0.88]
State (2,1): [-3.19 1.44 -0.47 -3.16]
State (2,2): [0. 0. 0. 0.]
State (2,3): [3.29 10. -0.36 1.77]
State (3,0): [-0.83 -0.65 -0.86 2.69]
State (3,1): [-0.47 1.61 -0.77 6.27]
State (3,2): [-0.19 2.29 0.57 9.42]
State (3,3): [0. 0. 0. 0.]